

09

Fajlovi, programi i korisnički unos

Python kao deo radnog toka: fajlovi i folderi, pokretanje drugih alata i ulaz od korisnika

Fajlovi i folderi: os i shutil

Zadatak	Funkcija
preimenuj ili premesti	<code>os.rename</code>
napravi folder	<code>os.mkdir</code>
napravi više foldera	<code>os.makedirs</code>
kopiraj fajl	<code>shutil.copy</code>
kopiraj folder	<code>shutil.copytree</code>

Zadatak	Funkcija
da li postoji?	<code>os.path.exists</code>
obriši fajl	<code>os.remove</code>
obriši prazan folder	<code>os.rmdir</code>
obriši folder sa sadržajem	<code>shutil.rmtree</code>
sadržaj foldera	<code>os.listdir</code>

```
import os
for file_name in os.listdir("."):
    if file_name.endswith(".dna"):
        print("reading sequences from " + file_name)
```

Isti kod radi na Linuxu, macOS-u i Windowsu. Putanje se ipak razlikuju; "." je trenutni folder.

Preimenovanje, premeštanje i kopiranje

```
import os
import shutil
# preimenovanje u istom folderu
os.rename("old.txt", "new.txt")
# ciljni folder mora da postoji
os.mkdir("/home/martin/python")
# isto ime, drugi folder: premeštanje
os.rename("/home/martin/biology/old.txt", "/home/martin/python/old.txt")
# više ugnežđenih foldera odjednom
os.makedirs("/a/long/path/with/lots/of/folders")
# kopiranje fajla i celog foldera
shutil.copy("/home/martin/original.txt", "/home/martin/copy.txt")
shutil.copytree("/home/martin/original_folder", "/home/martin/copy_folder")
```

rename radi i za foldere.

Putanje su sa autorovog računara (Linux). Za probu ih zamenite svojim.

Provera, brisanje i listanje

```
if os.path.exists("/home/martin/email.txt"):
    print("You have mail!")
# fajl, prazan folder, folder sa sadržajem
os.remove("/home/martin/unwanted_file.txt")
os.rmdir("/home/martin/empty")
shutil.rmtree("/home/martin/full")
# sadržaj foldera; "." je trenutni folder
for file_name in os.listdir("/home/martin"):
    print("one file name is " + file_name)
```

`os.path.exists` vraća True ili False, pa ide direktno u if.

`os.listdir` vraća listu imena, pa kroz nju prolazimo petljom.

Brisanje je trajno: fajlovi ne idu u korpu za otpatke.

Pokretanje spoljnih programa: subprocess

```
import subprocess
# pokreni program; izlaz ide na ekran
subprocess.call("/bin/date")
# sa opcijama komandne linije
subprocess.call("/bin/date +%B", shell=True)
# sačuvaj izlaz u promenljivoj
current_month = subprocess.check_output(
    "/bin/date +%B", shell=True, text=True)
```

Zašto?

Postojeće alate bi često bilo sporo ili teško pisati iznova.
Python ih povezuje u radni tok.

Proces i potproces

Pokrenut program je proces, a program koji pokrene drugi program pravi potproces (subprocess).

Izlaz programa obično ima novi red na kraju: `rstrip` pre dalje obrade.

call ispisuje, check_output vraća

```
subprocess.call("/bin/date")
```

```
Fri Jul 26 15:15:26 BST 2013
```

```
subprocess.call("/bin/date +%B", shell=True)
```

```
July
```

```
current_month = subprocess.check_output("/bin/date +%B",  
    shell=True, text=True)  
print(current_month.rstrip("\n"))
```

```
July
```

Izlaz programa koji pokrene `call` ide pravo na ekran. `check_output` ga vraća kao vrednost koju možemo da sačuvamo i obradimo.

Tri načina da podaci uđu u program

Ugrađeno u kod (hard-coded)

```
dna = "ATCGATCGTGACTAGCTACG"
```

Promena ulaza znači izmenu koda.

Interaktivno: input

```
accession = input("Enter the accession name")
```

Program čeka korisnika; uvek dobija string.

Komandna linija: sys.argv

```
$ python myprogram.py one two three
```

Najbolje za naučne radne tokove.

Saveti za korisnički unos

- Prikupite sav unos na početku, pre dugih izračunavanja
- Proverite unos čim stigne (validacija) i dajte korisniku drugu šansu
- Napišite jasna uputstva: program će koristiti i kolege, recenzenti ili vi sami za nekoliko meseci

Dve zamke interaktivnog unosa



Program tiho čeka

Posle dugog računanja program traži unos, a korisnik je otišao, misleći da sve radi. Program satima stoji.



Greška u kucanju na kraju

Program dugo radi, pa traži unos, a pogrešan unos ga obori. Sve mora ispočetka.

Rešenje: sav unos prikupite na početku i odmah ga proverite. Za rad bez nadzora, u lancu alata, koristite argumente komandne linije.

Argumenti komandne linije: sys.argv

```
# myprogram.py
import sys
print(sys.argv)
```

```
$ python myprogram.py one two three
```

```
['myprogram.py', 'one', 'two', 'three']
```

IZLAZ

sys.argv[0]

'myprogram.py'

ime programa

sys.argv[1]

'one'

prvi argument

sys.argv[2]

'two'

drugi argument

sys.argv[3]

'three'

treći argument

Svi argumenti su stringovi: broj dobijamo sa **int(sys.argv[1])**.

Sav ulaz je poznat odmah, pa ga proveravamo pre obrade.

Korak 1: pronadi fajlove i dužine sekvenci

```
import os
for file_name in os.listdir("."):
    if file_name.endswith(".dna"):
        print("reading sequences from " + file_name)
        dna_file = open(file_name)
        for line in dna_file:
            dna = line.rstrip("\n")
            length = len(dna)
            print("found a dna sequence with length " + str(length))
```

```
reading sequences from xag.dna
found a dna sequence with length 432
found a dna sequence with length 818
found a dna sequence with length 604
found a dna sequence with length 879
found a dna sequence with length 619
...
```

`endswith(".dna")`: čitamo samo fajlove sa pravom ekstenzijom.

`rstrip`: novi red nije baza, pa ga ne računamo u dužinu.

Korak 2: u koji opseg spada sekvenca?

```
# za svaku sekvencu: proveriti svaki opseg
for bin_lower in range(100, 1000, 100):
    bin_upper = bin_lower + 99
    if length >= bin_lower and length <= bin_upper:
        print("bin is " + str(bin_lower) + " to " + str(bin_upper))
```

PET NIVOA UVLAČENJA

petlja kroz fajlove

if: ime se završava sa .dna

petlja kroz sekvence u fajlu

petlja kroz opsege

if: sekvenca je u opsegu

sequence length is 432
bin is 400 to 499
sequence length is 818
bin is 800 to 899

Knjiga piše `length < bin_upper`, pa sekvenca od tačno 199 baza ne upada nigde. Ispravno je `<=`.

Korak 3: folderi i imena fajlova

```
import os
# po jedan folder za svaki opseg
for bin_lower in range(100, 1000, 100):
    bin_upper = bin_lower + 99
    bin_folder_name = str(bin_lower) + "_" + str(bin_upper)
    os.mkdir(bin_folder_name)
# brojač sekvenci za imena fajlova
seq_number = 1
```

NOVI FOLDERI

100_199 200_299 300_399
400_499 500_599 600_699
700_799 800_899 900_999

Ime foldera: donja granica, donja crta, gornja granica. Isto pravilo koristimo i pri upisu.

Imena fajlova nisu zadata: 1.dna, 2.dna, ... uz brojač koji raste posle svakog upisa.

Korak 4: upis u odgovarajući folder

```
# unutar petlje kroz .dna fajlove:
for line in dna_file:
    dna = line.rstrip("\n")
    length = len(dna)
    for bin_lower in range(100, 1000, 100):
        bin_upper = bin_lower + 99
        if length >= bin_lower and length <= bin_upper:
            bin_folder_name = str(bin_lower) + "_" + str(bin_upper)
            output_path = bin_folder_name + '/' + str(seq_number) + '.dna'
            output = open(output_path, "w")
            output.write(dna)
            output.close()
            seq_number = seq_number + 1
```

Putanja: folder, kosa crta, redni broj i .dna, npr. 400_499/1.dna.

Svaki fajl otvaramo, upisujemo i odmah zatvaramo.

Ovo je najveći program u knjizi do sada.

Plan u dve faze i deljenje na k-mere

1. Brojanje

Kroz sve sekvence, brojeve svih k-mera skupljamo u rečnik.



2. Ispis

Kroz rečnik, ispisujemo k-mere čiji je broj iznad praga.

```
test_dna = "ACTGTAGCTGTACGTAGC"
kmer_size = 4
for start in range(0, len(test_dna) - (kmer_size - 1), 1):
    kmer = test_dna[start:start+kmer_size]
    print(kmer)
```

ACTG	AGCT	TACG
CTGT	GCTG	ACGT
TGTA	CTGT	CGTA
GTAG	TGTA	GTAG
TAGC	GTAC	TAGC

15 preklapajućih 4-mera, čitano po kolonama.

Funkcija split_dna i njeni testovi

```
def split_dna(dna, kmer_size):  
    kmers = []  
    for start in range(0, len(dna) - (kmer_size - 1), 1):  
        kmer = dna[start:start+kmer_size]  
        kmers.append(kmer)  
    return kmers  
print(split_dna("AATGCTGCAT", 4))  
print(split_dna("AATGCTGCAT", 5))  
print(split_dna("AATGCTGCAT", 6))
```

Umesto ispisa, funkcija vraća listu k-mera. Ista sekvenca, različite dužine k-mera.

Funkciju je lakše testirati nego kod rasut po programu.

```
['AATG', 'ATGC', 'TGCT', 'GCTG', 'CTGC', 'TGCA', 'GCAT']  
['AATGC', 'ATGCT', 'TGCTG', 'GCTGC', 'CTGCA', 'TGCAT']  
['AATGCT', 'ATGCTG', 'TGCTGC', 'GCTGCA', 'CTGCAT']
```

Konačan program: argumenti i funkcija

```
import os
import sys
# argumenti komandne linije u brojeve
kmer_size = int(sys.argv[1])
count_cutoff = int(sys.argv[2])

def split_dna(dna, kmer_size):
    kmers = []
    for start in range(0, len(dna) - (kmer_size - 1), 1):
        kmer = dna[start:start + kmer_size]
        kmers.append(kmer)
    return kmers
```

Zadatak: u svim **.dna** fajlovima prebrojati k-mere zadate dužine i prikazati one koji se javljaju češće od zadanog praga.

```
$ python kmer_counting.py 6 25
```

int(): argumenti komandne linije su stringovi, a dužina i prag treba da budu brojevi

range: poslednji k-mer počinje **kmer_size - 1** mesta pre kraja, da bi bio ceo

Konačan program: brojanje kroz sve fajlove

```
kmer_counts = {}
for file_name in os.listdir("."):
    if file_name.endswith(".dna"):
        dna_file = open(file_name)
        for line in dna_file:
            dna = line.rstrip("\n")
            for kmer in split_dna(dna, kmer_size):
                current_count = kmer_counts.get(kmer, 0)
                kmer_counts[kmer] = current_count + 1

for kmer, count in kmer_counts.items():
    if count > count_cutoff:
        print(kmer + " : " + str(count))
```

IZLAZ ZA 6 25 (KNJIGA)

```
agagat : 26
agcggg : 26
gagtgg : 28
ccggtt : 26
ttctga : 26
ggacgt : 27
```

`get(kmer, 0) + 1`: rečnik kao brojač. Novi k-mer kreće od nule.

Rezime i vežbe

Šta smo naučili

- os i shutil: premeštanje, kopiranje, brisanje, listanje
- subprocess: call i check_output
- Ulaz: u kodu, preko input ili sys.argv
- Unos prikupite na početku i odmah ga proverite
- Greška može biti i van programa: fajl ne postoji, nema dozvole, spoljni program ne radi kako treba

Vežbe

Ulaz: **.dna** fajlovi sa sekvencama različitih dužina, po jedna u liniji.

1. Razvrstavanje: napravite 9 foldera (100–199 baza, 200–299 baza, ...) i svaku sekvencu upišite u poseban fajl u odgovarajućem folderu
2. Brojanje k-mera: dužina i prag kao argumenti komandne linije; prikažite k-mere koji se javljaju češće od praga

PREGLED

Šta sada umete

01 Pišete programe u editoru i pokrećete ih iz terminala

02 Isecate, brojite i menjate sekvence kao stringove

03 Čitate i pišete fajlove, npr. FASTA

04 Obradujete mnogo podataka listama i petljama

05 Pakujete kod u funkcije i testirate ga sa assert

06 Donosite odluke uslovima if, elif i else

07 Tražite fleksibilne šablone regularnim izrazima

08 Čuvate parove podataka u rečnicima

09 Upravlјate fajlovima, pokrećete alate, primate argumente

A T G A A C C C G G G T A A

Najbrži način da saznate šta neka funkcija radi: probajte je.

Rešavajte vežbe pre čitanja rešenja, isprobavajte varijacije i primenjujte alate na sopstvene podatke.